

BUILD-RULES-FOR-AI.md

Version : V1

Statut : Référence officielle de développement assisté par Intelligence Artificielle

Type : Build Blueprint

1. Objectif

Ce document définit les règles obligatoires que toute Intelligence Artificielle doit respecter lorsqu'elle génère, modifie ou refactorise du code pour Vitala.

Ces règles s'appliquent à tous les assistants IA utilisés sur le projet.

2. Champ d'application

Ces règles concernent notamment :

- Lovable
 - Claude Code
 - Cursor AI
 - GitHub Copilot
 - OpenAI Codex
 - Windsurf
 - toute autre IA générant du code.
-

3. Principe fondamental

Une IA n'invente jamais l'architecture.

Elle applique exclusivement :

- le VPS ;
- le Build Blueprint ;
- les contrats d'API ;
- le Domain Model ;
- les conventions officielles.

En cas d'ambiguïté, elle doit signaler le problème plutôt que de faire une hypothèse.

4. Règle de priorité documentaire

Lorsqu'une IA génère du code, elle respecte l'ordre suivant :

1. Vision Project Specification (VPS)
2. Build Blueprint
3. API Contracts
4. Database Specification
5. AI Development Rules
6. Demande de l'utilisateur

Aucune instruction ne peut contredire un niveau supérieur sans validation explicite.

5. Respect de l'architecture

L'IA doit respecter :

- Domain-Driven Design ;
- Clean Architecture ;
- API First ;
- Offline First ;
- Security by Design ;
- Modularité.

Aucun raccourci architectural n'est autorisé.

6. Construction des domaines

L'IA construit chaque domaine conformément à BUILD-DOMAINS.md.

Elle ne modifie jamais un domaine existant sans respecter son architecture et sa documentation.

7. Construction des APIs

Toute API générée doit :

- respecter son contrat officiel ;
- utiliser les cas d'utilisation du domaine ;
- appliquer les validations ;
- vérifier les permissions ;
- documenter les erreurs.

La logique métier dans les contrôleurs est interdite.

8. Construction de la base de données

L'IA ne crée jamais une table sans :

- domaine propriétaire ;
- migration ;
- politique RLS ;
- justification métier.

Toute modification du schéma passe par une migration.

9. Synchronisation

L'IA ne contourne jamais :

- l'Outbox ;
- les Checkpoints ;
- les règles de résolution des conflits.

Toute fonctionnalité Offline respecte BUILD-SYNC.md.

10. Interface utilisateur

L'IA ne place jamais de logique métier dans l'interface.

Les écrans consomment uniquement les APIs officielles.

Les composants restent réutilisables et conformes au Design System.

11. Moteurs d'intelligence

Les moteurs :

- fonctionnent en lecture ;
- produisent des analyses ;
- ne modifient jamais directement les domaines métier.

Toute recommandation doit rester explicable.

12. Documentation

Toute génération de code impactant :

- un domaine ;
- une API ;
- une table ;
- une architecture ;
- une règle métier ;

doit entraîner une mise à jour de la documentation concernée.

Le code et la documentation doivent rester synchronisés.

13. Refactoring

Lors d'un refactoring, l'IA doit :

- préserver le comportement fonctionnel ;
- conserver les contrats publics ;
- améliorer la lisibilité ;
- maintenir les tests ;
- éviter les régressions.

Un refactoring ne doit jamais introduire une nouvelle architecture.

14. Gestion des dépendances

L'IA ne crée jamais :

- de dépendances circulaires ;
- de dépendances non documentées ;
- de couplage fort entre domaines.

Toute nouvelle dépendance doit être justifiée.

15. Qualité du code

Le code généré doit être :

- lisible ;
- modulaire ;
- documenté lorsque nécessaire ;
- testable ;

- réutilisable.

La duplication est interdite.

16. Sécurité

Toute génération de code doit appliquer :

- validation des entrées ;
 - authentification ;
 - contrôle des permissions ;
 - politiques RLS ;
 - protection des données sensibles.
-

17. Tests

Avant de considérer une fonctionnalité terminée, l'IA doit vérifier :

- les tests unitaires ;
 - les tests d'intégration ;
 - les tests Offline lorsque nécessaires ;
 - les tests des APIs.
-

18. Interdictions

Une IA ne doit jamais :

- inventer une nouvelle architecture ;
 - ignorer un contrat d'API ;
 - contourner une règle métier ;
 - supprimer une validation ;
 - accéder directement à la base de données depuis l'interface ;
 - modifier un domaine sans mettre à jour la documentation.
-

19. Checklist obligatoire

Avant de terminer une tâche, l'IA vérifie systématiquement :

- conformité au VPS ;
- conformité au Build Blueprint ;
- conformité aux APIs ;
- conformité à la base de données ;

- conformité aux conventions ;
 - conformité aux règles de sécurité ;
 - conformité aux tests ;
 - mise à jour de la documentation.
-

20. Définition de réussite

Une tâche est considérée comme réussie uniquement lorsque :

- l'architecture est respectée ;
 - les tests sont validés ;
 - la documentation est synchronisée ;
 - aucun principe officiel n'est violé ;
 - le code est maintenable et évolutif.
-

21. Conclusion

BUILD-RULES-FOR-AI.md constitue la référence officielle de toute génération de code assistée par Intelligence Artificielle dans le projet Vitala.

Il garantit que toutes les IA produisent un code cohérent, sécurisé, conforme à l'architecture et durable dans le temps.